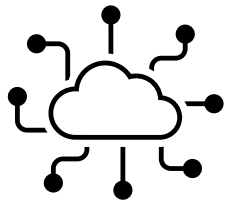
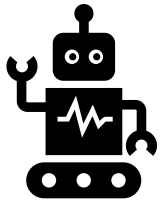
ACTIVIDAD INTEGRAL 1

Búsqueda y Ordenamiento

EL ATAQUE DE LAS BOTNETS





Se nos provee un archivo mostrando el tráfico dentro de una empresa que tras ser infectada por un una botnet fue utilizada en un ataque DDoS. A través de diversas actividades integradoras procesarás y analizarás dicho archivo buscando resolver algunas preguntas clave:

- ¿Cuál fue el primer host infectado?
- ¿Cuál es la ip del bot master?
- ¿Contra qué dominio se realizó el DDoS attack?

Para comenzar, debemos entender el formato de los datos. La bitácora de actividad está en formato csv (valores separados por comas) y cuenta con el siguiente formato:

- Fecha (dd/mm/aaaa), hora (hh:mm:ss), ip origen, puerto origen, dominio origen, ip destino, puerto destino, dominio destino.

Donde cada renglón representa un evento registrado en la bitácora (una petición de conexión entre un origen y un destino).

El carácter – se utiliza para distinguir valores que no aplican (por ejemplo una ip no asociada a un dominio, o una conexión que no especifica un puerto o un nodo que aún no tiene asignada una dirección ip).

Lamentablemente, la bitácora provista fue incorrectamente generada por el administrador de la red, por lo que los datos parecen estar desordenados. En esta primera actividad nos concentraremos en ordenarlos a partir de la fecha y hora de cada evento registrado. Así mismo, generaremos una utilidad que nos permita filtrar eventos a partir de un rango dado.

INSTRUCCIONES

- En esta primera actividad, deberás escribir un programa que:
  - Lea el archivo provisto que le corresponde a tu equipo y almacena su información en un vector
    - Cada elemento del vector deberá ser una estructura conteniendo el conjunto de valores que conforman cada línea del archivo
  - Ordene el vector de forma ascendente a partir de la fecha y hora
  - Almacene el resultado del ordenamiento en un archivo
  - Solicite al usuario fechas de inicio y fin de la búsqueda y despliegue todos los registros correspondientes al periodo entre inicio (inclusivo) y fin (exclusivo)

Adicionalmente, realiza una investigación sobre los algoritmos de ordenamiento y búsqueda utilizados en la resolución de esta actividad, así como su complejidad (revisa la documentación correspondiente).



TECH SUPPORT

STRUCTS

Tipo de dato agregativo definido por el programador

Permite agrupar un conjunto de valores relacionados en un mismo “contenedor”

A diferencia de un arreglo, los valores pueden ser de distintos tipos y poseer nombres propios (miembros)

Similar a una clase, pero sin métodos

Declarados utilizando la palabra reservada struct seguido del nombre y los miembros entre { }

- Necesario poner ; después de }

Miembros pueden ser leídos y escritos mediante notación .

Más información: https://www.learncpp.com/cpp-tutorial/47-structs/

```
struct Employee
{
    short id;
    int age;
    double wage;
};
```

```
Employee joe; // create an Employee struct for Joe
joe.id = 14; // assign a value to member id within struct joe
joe.age = 32; // assign a value to member age within struct joe
joe.wage = 24.15; // assign a value to member wage within struct joe

Employee frank; // create an Employee struct for Frank
frank.id = 15; // assign a value to member id within struct frank
frank.age = 28; // assign a value to member age within struct frank
frank.wage = 18.27; // assign a value to member wage within struct frank
```

SOBRECARGA DE OPERADORES

C++ permite la sobrecarga de todos sus operadores

Definición de un operador dado por el método operator correspondiente

- Por ejemplo, < corresponde al método operator<

Para sobrecargar un operador, necesitamos sobrecargar el método correspondiente con los tipos de datos que queremos usar

Los parámetros que reciben las funciones deben ser:

- Referencias (&): más eficiente pues evitamos realizar una copia del elemento
- Constante (const): Realizar una comparación jamás debe alterar el valor de los operandos, const garantiza esto

Más información: https://www.learncpp.com/cpp-tutorial/96-overloading-the-comparison-operators/

```
bool operator== (const Car &c1, const Car &c2)
{
    return (c1.m_make == c2.m_make &&
            c1.m_model == c2.m_model);
}
```

```
bool operator< (const Cents &c1, const Cents &c2)
{
    return c1.m_cents < c2.m_cents;
}
```

DATE & TIME

Diferentes maneras de representar fecha+hora en C++

Librería <ctime>

struct tm

- Permite definir una fecha+hora como una estructura con miembros para cada elemento
  - Año, Mes, Día, Hora, Minuto, Segundo, Etc.

time_t

- Representa fecha+hora como un valor entero (número de segundos desde UNIX time)

mktime

- Permite convertir tm a time_t

https://cplusplus.com/reference/ctime/

```cpp
#include <ctime>
#include <iostream>

int main() {
    struct tm tm1 = {0}; // Initialize to all zeros
    tm1.tm_year = 2025 - 1900; // Years since 1900
    tm1.tm_mon = 8; // September (0-indexed)
    tm1.tm_mday = 25; // Day of the month

    struct tm tm2 = {0};
    tm2.tm_year = 2025 - 1900;
    tm2.tm_mon = 8;
    tm2.tm_mday = 26;

    time_t t1 = mktime(&tm1);
    time_t t2 = mktime(&tm2);

    // Compare the time_t values
    if (t1 < t2) {
        std::cout << "tm1 is earlier than tm2" << std::endl;
    } else if (t1 > t2) {
        std::cout << "tm1 is later than tm2" << std::endl;
    } else {
        std::cout << "tm1 and tm2 represent the same time" << std::endl;
    }
```

<ALGORITHM>

Header de la librería estándar de C++ que incluye diversos métodos que implementan algoritmos de uso común en rangos.

En particular:

- std::sort(Iterator first, Iterator last) – Ordena de forma ascendente el vector dado, en el rango especificado (de first a last). Los elementos del vector deben ser comparables mediante el operador < (es decir, el operador < debe estar sobrecargado para el tipo de dato contenido en el vector)
  - http://www.cplusplus.com/reference/algorithm/sort/

- std::find_if(Iterator first, Iterator last, UnaryPredicate pred) – Encuentra la primera ocurrencia de un objeto en el rango especificado (de first a last) que evalue a verdadero con el predicado y regresa un iterador apuntando a dicho elemento.
  - https://cplusplus.com/reference/algorithm/find_if/

ITERADORES

```cpp
#include <iostream>
#include <vector>

int main ()
{
  std::vector<int> myvector;
  for (int i=1; i<=5; i++) myvector.push_back(i);

  std::cout << "myvector contains:";
  for (std::vector<int>::iterator it = myvector.begin() ; it != myvector.end(); ++it)
    std::cout << ' ' << *it;
  std::cout << '\n';

  return 0;
}
```

Objetos usados para navegar de forma más sencilla colecciones de datos

Cada colección de datos cuenta con su propia implementación de iteradores relevantes

Para vector:

- Tipo de dato std::vector<T>::iterator, donde T es el tipo de dato guardado en el vector
- Métodos relevantes:
  - begin(): regresa un iterador al inicio del vector
  - end(): regresa un iterador al final del vector (uno después del último elemento)
- Un iterador it puede manipularse utilizando operadores:
  - it++: avanza el iterador a la siguiente posición del vector
  - it--: regresa el iterador a la posición previa del vector
  - *it: regresa el valor contenido en la posición del vector
  - it < otro_it: regresa verdadero si it es una posición previa a otro_it
- http://www.cplusplus.com/reference/vector/vector/begin/
- http://www.cplusplus.com/reference/vector/vector/end/

Y ALGO MÁS…

Lectura y escritura de archivos

Lectura y escritura de archivos a través de streams

- Muy similar a lectura y escritura a consola

http://www.cplusplus.com/doc/tutorial/files/

Vector

Arreglos de tamaño dinámico

- Localidades contiguas de memoria con manejo automático para crecer o hacer más pequeño el arreglo

http://www.cplusplus.com/reference/vector/vector/